

コマ 1: 環境構築 + RV64 手書きアセンブリ

今日のゴール

qemu 上で手書き RV64 アセンブリを実行し、終了コード 42 を確認する。あわせて、教員提供の Lexer/Parser で $1+2*3$ の AST を表示できることも確認する。

この回ではコード生成は行わない。まずは RISC-V アセンブリを手で書いて、アセンブル・リンク・実行の流れを理解する。

1 開発環境

- Docker 環境 (docker/rv64/) を推奨。ネイティブ実行も許可する
- 必要なツール: riscv64-linux-gnu-gcc, qemu-riscv64

```
bash docker/rv64/run.sh
```

2 手書きアセンブリを動かす

2.1 Step 1: hello.s を書く

配布される hello.s は終了コード 0 を返す未完成 starter である。0 を 42 に変更して、以下の内容にする。

```
.global main
main:
    addi    a0, zero, 42    # 戻り値 = 42
    ret
```

2.2 Step 2: コンパイルして実行

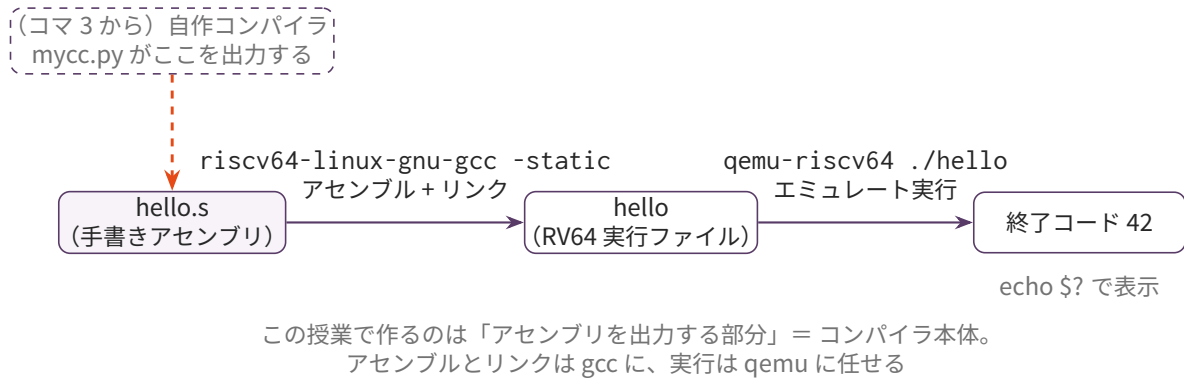


図 1 hello.s が実行されて終了コードになるまでの流れ

```

riscv64-linux-gnu-gcc -static hello.s -o hello
qemu-riscv64 ./hello
echo $? # → 42
  
```

42 が表示されれば成功。

この演習で作るのは、図の hello.s にあたる「アセンブリを出力する部分」= コンパイラ本体である。コマ 3 以降は、この部分を自作コンパイラ mycc.py が出力するようになる。

2.3 Step 3: 確認スクリプト

```
python3 sessions/01_environment/check.py
```

2.4 Step 4: もっと複雑な計算

ファイル	内容	期待値
tests/hello.s	addi a0, zero, 42 で戻り値 42 を返す	42
tests/sub.s	50 + (-8) で減算相当	42
tests/three_numbers.s	10 + 20 + 12 で 3 数の和	42

これら 2 件は完成済みの命令例であり、編集対象ではない。addi / li / add などの基本命令

の組み合わせを確認する。

3 スキャフォールドの動作確認

教員提供の Lexer/Parser が AST を正しく構築できることを確認する。

```
python3 scaffold/parse_viewer.py sessions/02_interpreter/tests/add_mul.c
```

以下のような S 式が表示されれば成功。

```
(program
  (funcdef "main" :type int (params)
    (block
      (return
        (add (num 1)
              (mul (num 2) (num 3)))))))
```

`1 + 2 * 3` が `(add (num 1) (mul (num 2) (num 3)))` のように、掛け算が先にまとめられている（演算子の優先順位が正しく反映されている）ことを確認する。

4 編集するファイル

- `hello.s`

最初に `addi a0, zero, 0` を終了コード 42 を返す命令へ変更する。

5 テスト

```
python3 sessions/01_environment/check.py
```

このコマンドは、編集した `hello.s`、完成済みの追加例 2 件、`1 + 2 * 3` の AST 構造をまとめて確認する。

6 次回予告

今回は AST を走査して式を評価する「インタープリター」を書く。コード生成に入る前に、AST の各ノードが何を意味するかを体で理解する。